

Introduction to
**Retrieval-Augmented Generation
(RAG)**

Architecture & Purpose

Huzaifa Arshad

May 2026

1. Introduction

Large Language Models (LLMs) such as GPT-4 and Claude have demonstrated remarkable capabilities in language understanding and generation. However, these models suffer from a fundamental limitation: their knowledge is frozen at the time of training. They cannot access real-time information, proprietary databases, or domain-specific documents that were not part of their training corpus.

Retrieval-Augmented Generation (RAG) is a powerful architectural pattern that addresses this limitation by combining the generative capabilities of LLMs with dynamic information retrieval from external knowledge sources. Introduced by Lewis et al. (2020) at Meta AI Research, RAG has quickly become one of the most widely adopted techniques in enterprise AI deployments.

This report provides a comprehensive overview of RAG — its motivation, core architecture, key components, workflow, variants, use cases, and current limitations. It serves as a foundational reference for implementing a RAG-based Q&A system during this internship.

2. What is Retrieval-Augmented Generation?

RAG is a hybrid AI framework that augments the input prompt of a generative language model with contextually relevant information retrieved from an external knowledge base at inference time. Instead of relying solely on parametric memory (weights learned during training), RAG grounds the model's responses in retrieved non-parametric memory (external documents).

2.1 The Core Problem RAG Solves

Traditional LLMs face the following challenges that RAG directly addresses:

- **Knowledge cutoff:** LLMs cannot answer questions about events after their training date.
- **Hallucination:** Without grounding, models may generate plausible but factually incorrect information.
- **Lack of domain specificity:** General-purpose LLMs lack access to proprietary or specialized knowledge bases.
- **Transparency:** Pure generation does not provide source citations, making verification difficult.
- **Cost of retraining:** Updating an LLM with new information requires expensive full or fine-tune retraining.

RAG solves all of these by dynamically fetching relevant documents at query time and injecting them into the LLM's context window — enabling up-to-date, grounded, and verifiable responses.

3. RAG Architecture

The RAG architecture consists of two primary phases: an offline Indexing Phase and an online Query Phase. Together, these form a complete pipeline from raw documents to generated answers.

3.1 Indexing Phase (Offline)

This phase is responsible for ingesting and preparing the knowledge base. It involves the following steps:

Step	Component	Description
1	Document Loading	Raw documents (PDFs, web pages, databases, etc.) are ingested using document loaders.
2	Text Chunking	Documents are split into smaller, overlapping chunks to fit within the embedding model's context window.
3	Embedding Generation	Each chunk is converted into a dense vector representation using an embedding model (e.g., text-embedding-ada-002).
4	Vector Storage	Embeddings are stored in a vector database (e.g., FAISS, Pinecone, Weaviate, Chroma) for efficient similarity search.

3.2 Query Phase (Online)

When a user submits a query, the following steps are executed in real time:

- **Query Embedding:** The user's query is converted into a vector using the same embedding model.
- **Similarity Search:** The query vector is compared against all stored document vectors using cosine similarity or dot product.
- **Top-K Retrieval:** The K most relevant document chunks are retrieved from the vector store.
- **Prompt Construction:** Retrieved chunks are injected into a structured prompt along with the original query.
- **LLM Generation:** The augmented prompt is passed to the LLM, which generates a grounded, contextual response.

- **Response Delivery:** The final answer (with optional citations) is returned to the user.

4. Key Components of a RAG System

4.1 Embedding Models

Embedding models convert text into high-dimensional vector representations that capture semantic meaning. Texts with similar meaning have vectors that are close together in the embedding space. Popular choices include OpenAI's text-embedding-ada-002, HuggingFace's sentence-transformers (e.g., all-MiniLM-L6-v2), and Cohere's embed-v3.

4.2 Vector Databases

Vector databases are purpose-built to store and query high-dimensional embeddings efficiently. They support approximate nearest neighbor (ANN) search algorithms such as HNSW and IVF-PQ for fast retrieval at scale.

Vector DB	Type	Key Feature
FAISS	In-memory / Local	Highly optimized by Meta, great for prototyping
Chroma	Local / Embedded	Easy to set up, ideal for development
Pinecone	Cloud-managed	Fully managed, production-ready, scalable
Weaviate	Self-hosted / Cloud	Supports hybrid search (dense + sparse)
Qdrant	Self-hosted / Cloud	High performance, supports filtering

4.3 Chunking Strategy

How documents are split into chunks significantly impacts retrieval quality. Common strategies include fixed-size chunking (e.g., 512 tokens with 50-token overlap), recursive character splitting, semantic chunking based on sentence boundaries, and document-structure-aware splitting (e.g., by headings).

4.4 Large Language Model (LLM)

The LLM is the generative component that synthesizes retrieved information into a coherent response. It can be an API-based model (GPT-4, Claude, Gemini) or an open-source model (LLaMA 3, Mistral, Falcon) deployed locally.

5. RAG Variants & Advanced Techniques

The basic RAG pipeline has evolved into several advanced variants to address its limitations:

Variant	Description	Use Case
Naive RAG	Simple retrieve-then-generate pipeline with single retrieval step.	Prototyping, simple Q&A
Advanced RAG	Adds query rewriting, re-ranking, and hybrid search for better retrieval.	Production Q&A systems
Modular RAG	Flexible pipeline with swappable components for retrieval, fusion, and generation.	Complex enterprise systems
Self-RAG	LLM decides when to retrieve and critiques its own outputs.	High-accuracy tasks
Graph RAG	Uses knowledge graphs alongside vector retrieval for structured reasoning.	Complex multi-hop reasoning
Agentic RAG	Combines RAG with AI agents that can take multi-step retrieval actions.	Research assistants, copilots

6. Real-World Use Cases

RAG has been widely adopted across industries due to its practicality and effectiveness:

- **Enterprise Knowledge Management:** Internal chatbots that answer HR, legal, or IT questions by retrieving from internal documentation.
- **Customer Support Automation:** AI agents that retrieve product manuals, FAQs, and past tickets to resolve customer queries accurately.
- **Medical & Legal Q&A:** Domain-specific assistants that ground answers in verified clinical guidelines or legal statutes.
- **Code Assistance:** Developer copilots that retrieve relevant code snippets, API documentation, or GitHub issues.
- **Financial Analysis:** Systems that retrieve real-time filings, reports, and news to answer analyst queries.
- **Educational Tutoring:** Personalized tutors that retrieve relevant textbook passages to answer student questions with citations.

7. Limitations and Challenges

Despite its advantages, RAG is not without challenges:

Challenge	Description	Mitigation
Retrieval Failure	Relevant chunks may not be retrieved if the query is ambiguous or poorly formulated.	Query rewriting, HyDE, hybrid search
Chunking Quality	Poor chunking can split related information, degrading retrieval accuracy.	Semantic chunking, document-aware splitting
Context Window Limits	LLMs have limited context windows; too many chunks may exceed limits.	Re-ranking, selective retrieval, compression
Latency	Adding a retrieval step increases end-to-end latency compared to pure generation.	Caching, async retrieval, smaller embeddings
Hallucination	LLMs may still hallucinate even with retrieved context if not properly prompted.	Strict prompting, faithfulness checks, Self-RAG
Knowledge Freshness	Vector index must be kept up to date as the knowledge base changes.	Incremental indexing, real-time connectors

8. Conclusion

Retrieval-Augmented Generation represents a paradigm shift in how we deploy large language models in production. By grounding LLM responses in dynamically retrieved, up-to-date external knowledge, RAG systems achieve higher factual accuracy, better domain specificity, and improved trustworthiness compared to pure generative approaches.

RAG is not a silver bullet — careful attention to chunking strategy, embedding quality, retrieval accuracy, and prompt engineering is required. However, when implemented thoughtfully, it enables organizations to deploy reliable, context-aware AI systems at scale without the prohibitive cost of continual model retraining.